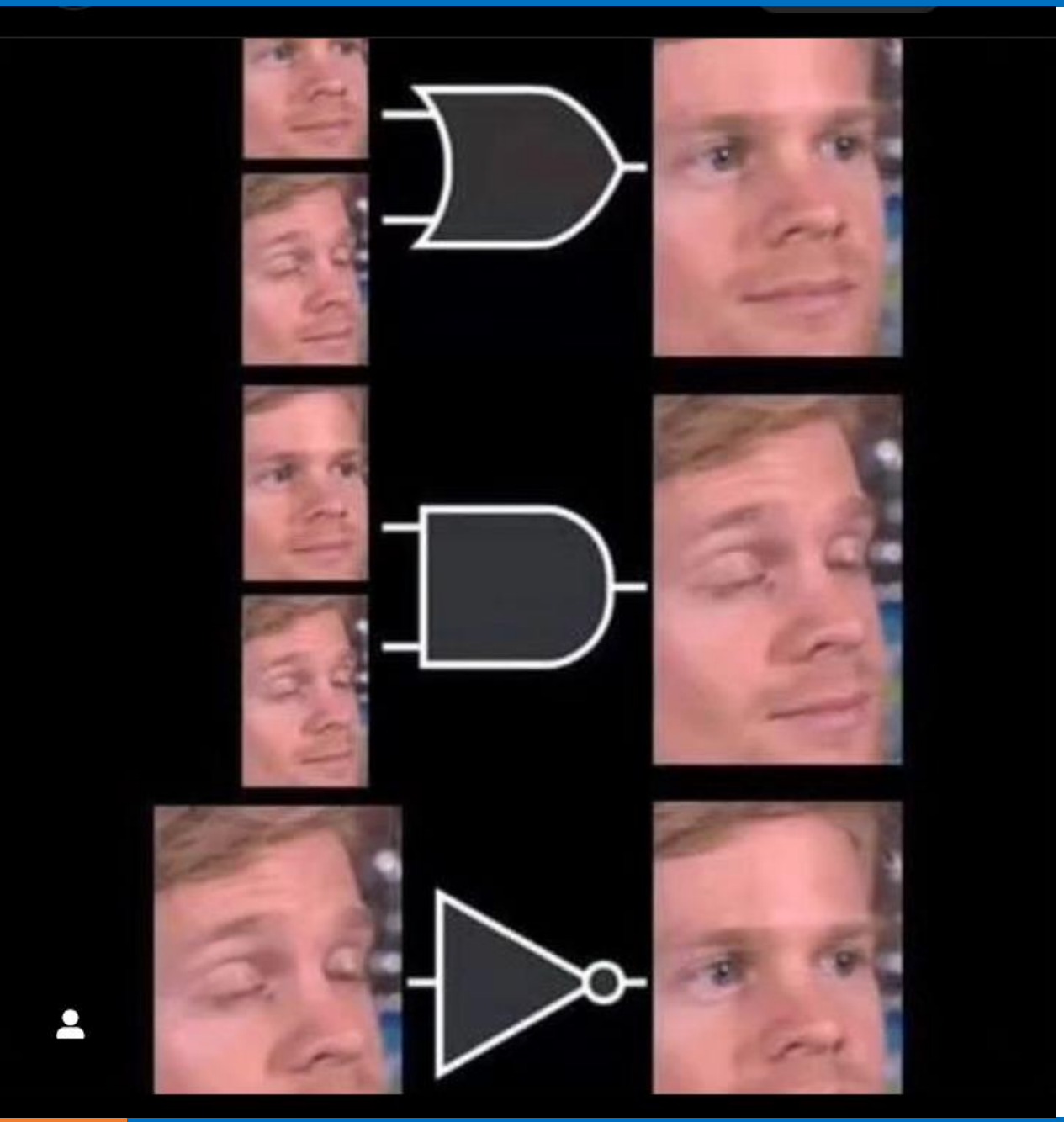


UNIVERSITAS DIPONEGORO

Gerbang Logika

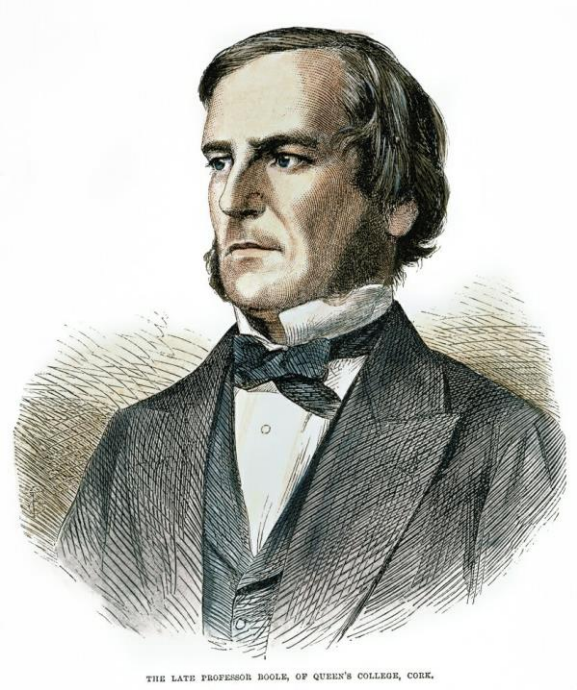
VTE2624203 – Elektronika Digital

Dhani Nur Indra Syamputra, S.Si., M.Sc.

Introduction

UNIVERSITAS DIPONEGORO

- The binary number system uses only two digits, 1 and 0, so it is perfect for representing logical relationships.
- Life is full of examples of circumstances that are in one state or another.
- In 1854, a mathematician named George Boole wrote *An Investigation of the Laws of Thought*, in which he described the way we make logical decisions based on true or false circumstances.
- The methods he described are referred to today as ***Boolean logic***, and the system of using symbols and operators to describe these decisions is called ***Boolean algebra***.
- Boolean algebra uses symbols to represent a logical expression that has one of two possible values: true or false.
- The logical expression might be *door is closed*, *button is pressed*, or *fuel is low*.
- Boolean algebra is not only used as a tool for analysis and simplification of logic systems. It can also be *used as a tool to create a logic circuit* that will produce the desired input/output relationship.



George Boole
(1815-1864)

Boolean Constants and Variables

UNIVERSITAS DIPONEGORO

- Boolean algebra differs in a major way from ordinary algebra because Boolean constants and variables are allowed to have only two possible values, 0 or 1.
- Thus, Boolean 0 and 1 do not represent actual numbers but instead represent the state of a voltage variable, or what is called its ***logic level***.

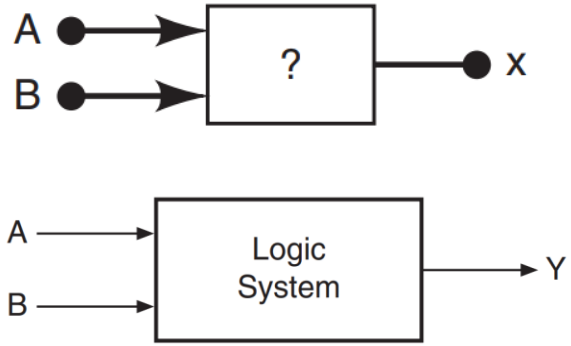
Logic 0	Logic 1
False	True
Off	On
LOW	HIGH
No	Yes
Open switch	Closed switch

- ***Boolean algebra*** is a means for expressing the relationship between a logic circuit's inputs and outputs.
- The inputs are considered logic variables whose logic levels at any time determine the output levels.
- We shall use letter symbols to represent logic variables. For example, the letter A might represent a certain digital circuit input or output, and at any time we must have either A = 0 or A = 1: if not one, then the other.
- Because only two values are possible, Boolean algebra is relatively easy to work with compared with ordinary algebra. In Boolean algebra there are only three basic operations: *OR, AND, and NOT*, called ***logic operations***.

The Truth Table

UNIVERSITAS DIPONEGORO

- A ***truth table*** is a means for describing how a logic circuit's output depends on the logic levels present at the circuit's inputs.
- The table lists all possible combinations of logic levels present at inputs *A* and *B*, along with the corresponding output level *x*.
- The number of input combinations will equal 2^N for an *N*-input truth table.
- The list of all possible input combinations follows the binary counting sequence, and so it is an easy matter to write down all of the combinations without missing any.



Inputs → A, B; Output → x

A	B	x
0	0	1
0	1	0
1	0	1
1	1	0

A	B	C	x
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

A	B	C	D	x
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

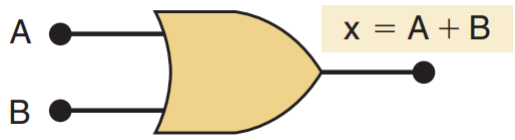
OR Operation & Gate

UNIVERSITAS DIPONEGORO

- An example of OR Operation can be found in the kitchen oven. The light inside the oven should turn on if either the *oven light switch is on* OR if *the door is opened*.
- The Boolean expression for the OR operation is

$$x = A + B$$

  In this expression, the + sign does not stand for ordinary addition; it stands for the OR operation.
- In digital circuitry, an ***OR gate*** is a circuit that has two or more inputs and whose output is equal to the OR combination of the inputs.



OR

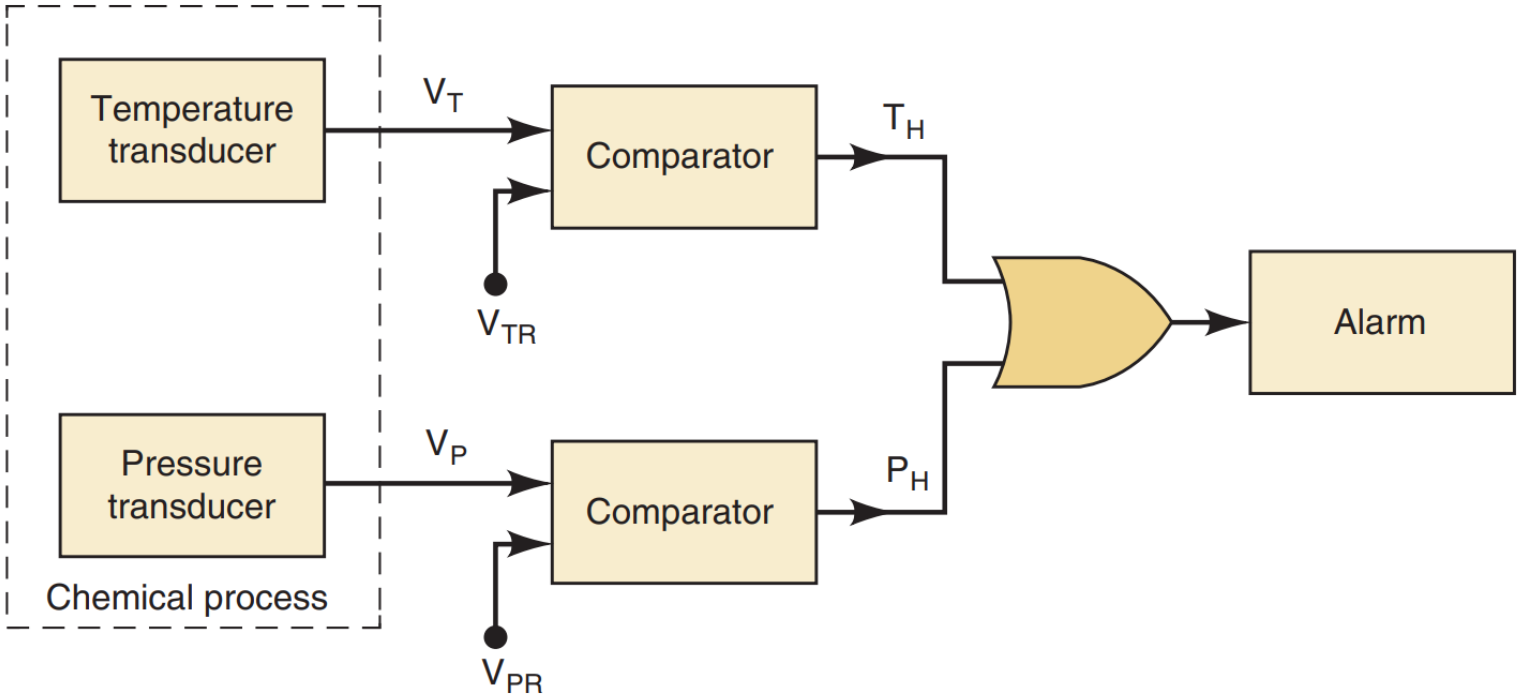
A	B	x = A + B
0	0	0
0	1	1
1	0	1
1	1	1

1. The OR operation produces a result (output) of 1 whenever any input is a 1. Otherwise the output is 0.
2. An OR gate is a logic circuit that performs an OR operation on the circuit's inputs.
3. The expression $x = A + B$ is read as "*x equals A OR B*".

Example 1

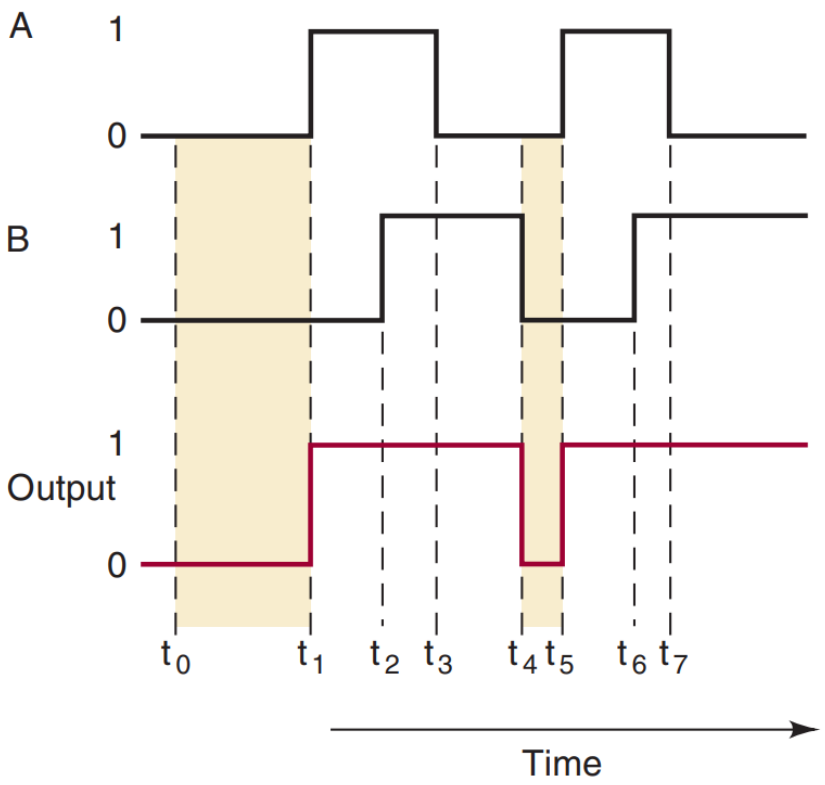
UNIVERSITAS DIPONEGORO

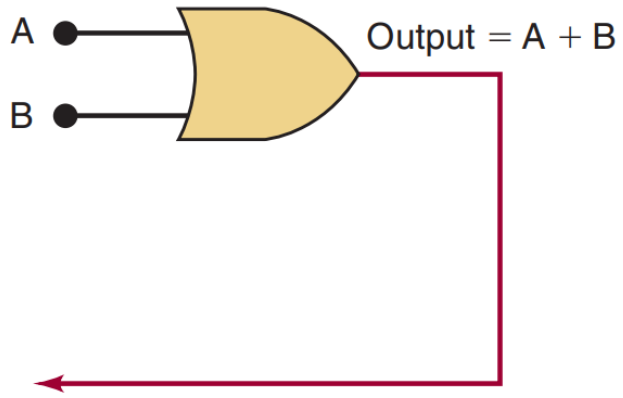
- In many industrial control systems, it is required to activate an output function whenever any one of several inputs is activated. For example, in a chemical process it may be desired that an alarm be activated whenever the process temperature exceeds a maximum value *or* whenever the pressure goes above a certain limit.



Example 2

- Determine the OR gate output in figure below. The OR gate inputs A and B are varying according to the timing diagrams shown. For example, A starts out LOW at time t_0, goes HIGH at t_1, back to LOW at t_3, and so on.





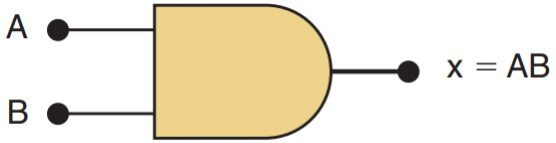
AND Operation & Gate

UNIVERSITAS DIPONEGORO

- An example of the use of AND logic, consider a typical clothes dryer. It is drying clothes (heating, tumbling) only *if the timer is set above zero* AND *the door is closed*.
- The Boolean expression for the AND operation is

$$x = A \cdot B$$

  In this expression, the · sign stands for the Boolean AND operation and not the multiplication operation. However, the AND operation on Boolean variables operates the same as ordinary multiplication.
- the output of AND Gate will go HIGH **only** when all inputs are HIGH.



AND gate

AND

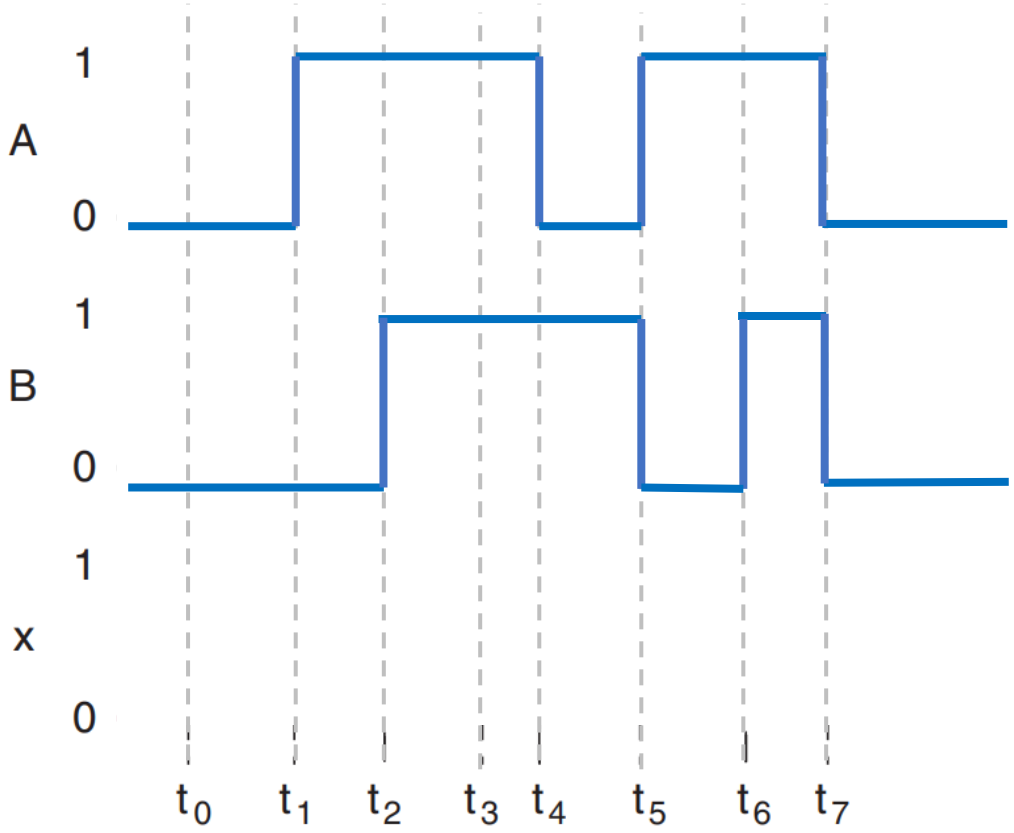
A	B	x = A•B
0	0	0
0	1	0
1	0	0
1	1	1

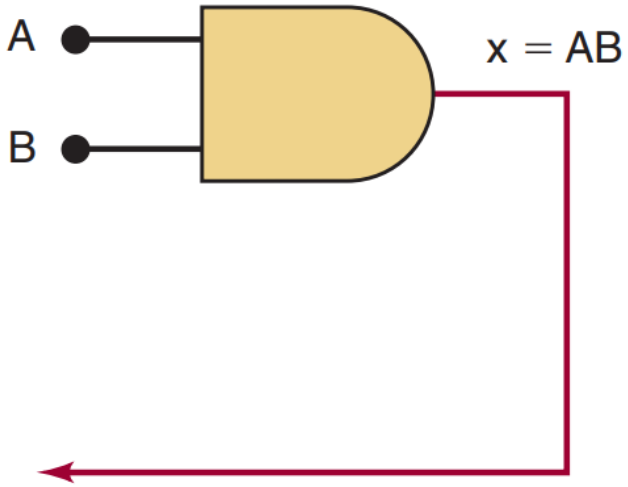
1. The AND operation is performed the same as ordinary multiplication of 1s and 0s.
2. An AND gate is a logic circuit that performs the AND operation on the circuit's inputs.
3. An AND gate output will be 1 only for the case when all inputs are 1; for all other cases, the output will be 0.
4. The expression x = AB is read as "*x equals A AND B*".

Exercise 1

UNIVERSITAS DIPONEGORO

- Determine the output *x* from the AND gate for the given input waveforms.





NOT Operation & Gate

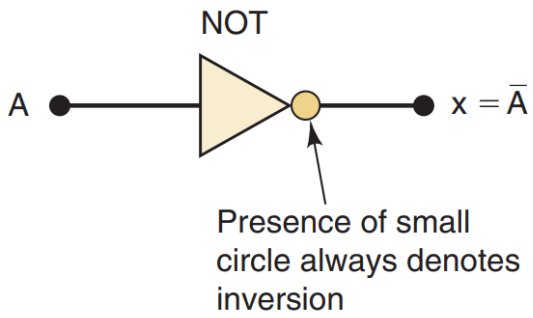
UNIVERSITAS DIPONEGORO

- The NOT operation is unlike the OR and AND operations because it can be performed on a single input variable.
- For example, if the variable *A* is subjected to the NOT operation, the result *x* can be expressed as

$$x = \overline{A} = A'$$

where the overbar represents the NOT operation.

- The NOT operation is also referred to as ***inversion*** or ***complementation***.
- NOT circuit is more commonly called an INVERTER, which always has only *a single input*, and its output logic level is always opposite to the logic level of the input.



NOT

A	$x = \overline{A}$
0	1
1	0

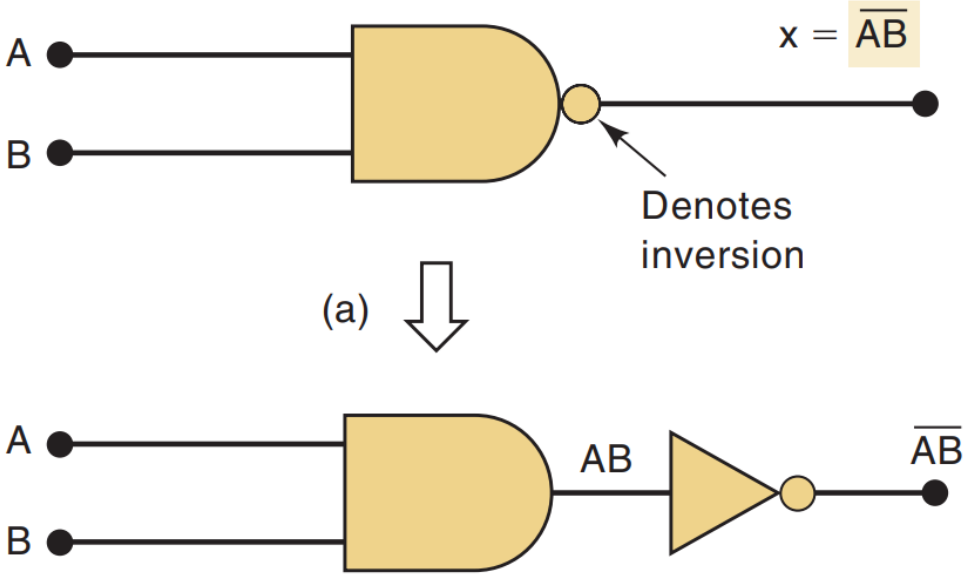
NAND Gate

UNIVERSITAS DIPONEGORO

- The **NAND** operates like an **AND** gate followed by an **INVERTER**.
- The output expression for the NAND gate is,

$$x = \overline{A \cdot B}$$

- The output of a NAND gate is a logic '0' when all its inputs are a logic '1'. For all other input combinations, the output is a logic '1'.



		AND	NAND
A	B	AB	$\overline{AB}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

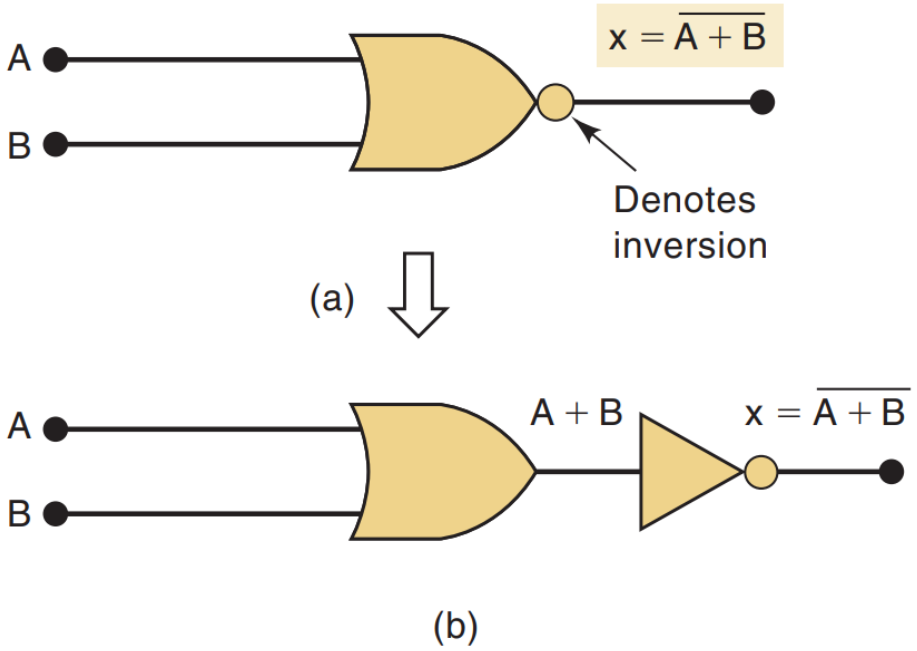
NOR Gate

UNIVERSITAS DIPONEGORO

- The **NOR** gate operates like an **OR** gate followed by an **INVERTER**.
- The output expression for the NOR gate is

$$x = \overline{A + B}$$

- The output of a NOR gate is a logic '1' when all its inputs are logic '0'. For all other input combinations, the output is a logic '0'.



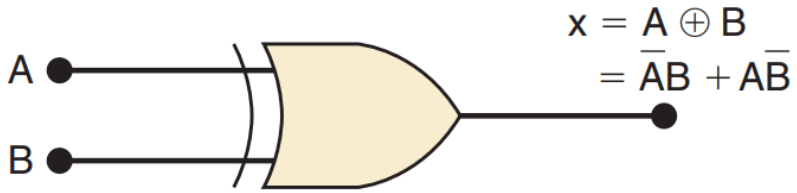
		OR	NOR
A	B	A + B	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Exclusive-OR Gate

UNIVERSITAS DIPONEGORO

- The EXCLUSIVE-OR gate, commonly written as EX-OR (XOR) gate, is a two-input, one-output gate.
- The output of an EX-OR gate is a logic '***1***' when the inputs are ***unlike*** and a logic '***0***' when the inputs are ***like***.
- The output of a two-input EX-OR gate is expressed by

$$x = A \oplus B = \bar{A}B + A\bar{B}$$



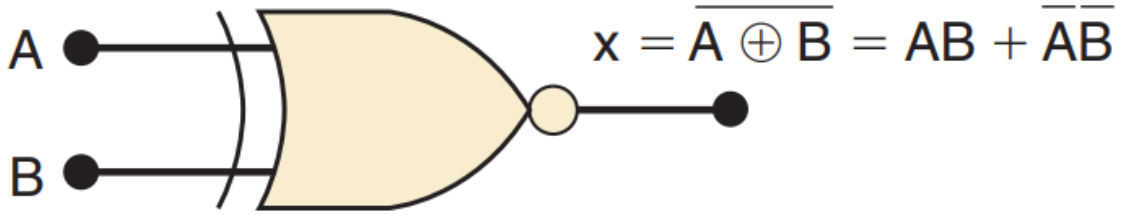
A	B	x
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive-NOR Gate

UNIVERSITAS DIPONEGORO

- The exclusive-NOR circuit (abbreviated **XNOR**) operates completely *opposite* to the XOR circuit.
- The output of a two-input EX-NOR gate is a logic ***'1'*** when the inputs are **like** and a logic ***'0'*** when they are **unlike**.
- The output expression for XNOR output is,

$$x = \overline{A \oplus B} = AB + \overline{A}\,\overline{B}$$



A	B	x
0	0	1
0	1	0
1	0	0
1	1	1

Boolean Algebra

Next Meeting...